COMPUTER ARCHITECTURE

Pohandoi Aslamzai

Basic Input/output

Lecture Outline

- External Devices
- I/O Modules
- Programmed I/O
- An Example of a RISC-Style I/O Program
- An Example of a CISC-Style I/O Program

I/O System Design Tool

- In addition to the processor and a set of memory modules
  - Set of I/O modules
- Each module is an interface to the system bus or
  - Central switch and controls one or more peripheral devices.
- I/O module is not a simply a set of mechanical connectors
  - Rather, it contains logic for performing a communication function between the peripheral and the bus.

I/O System Design Tool

- Why one does not connect peripherals directly to the system bus
  - Wide variety of peripherals with various methods of operation.
  - Data transfer rate of peripherals is often much slower and some of them faster.
  - Peripherals often use different data formats and word lengths.
- Need I/O modules

I/O System Design Tool

- This module has two functions
  - Interface to the processor and memory via the system bus or central switch.
  - Interface to one or more peripheral devices by tailored data links.

Cont.

Load R2, DATAIN

- reads the data from the DATAIN register and loads them into processor register R2. Similarly, the instruction

Store R2, DATAOUT

- sends the contents of register R2 to location DATAOUT, which is a register in an output device.

Device interface

- An I/O device is connected to the interconnection network by using a circuit, called the *device interface*
- which provides the means for data transfer and for the exchange of status and control information needed to facilitate the data transfers and govern the operation of the device.
- The interface includes some registers that can be accessed by the processor

Program-controlled I/O

- Consider a task that reads characters typed on a keyboard, stores these data in the memory, and displays the same characters on a display screen.
- A simple way of implementing this task is to write a program that performs all functions needed to realize the desired action. This method is known as *program-controlled I/O*.

Cont..

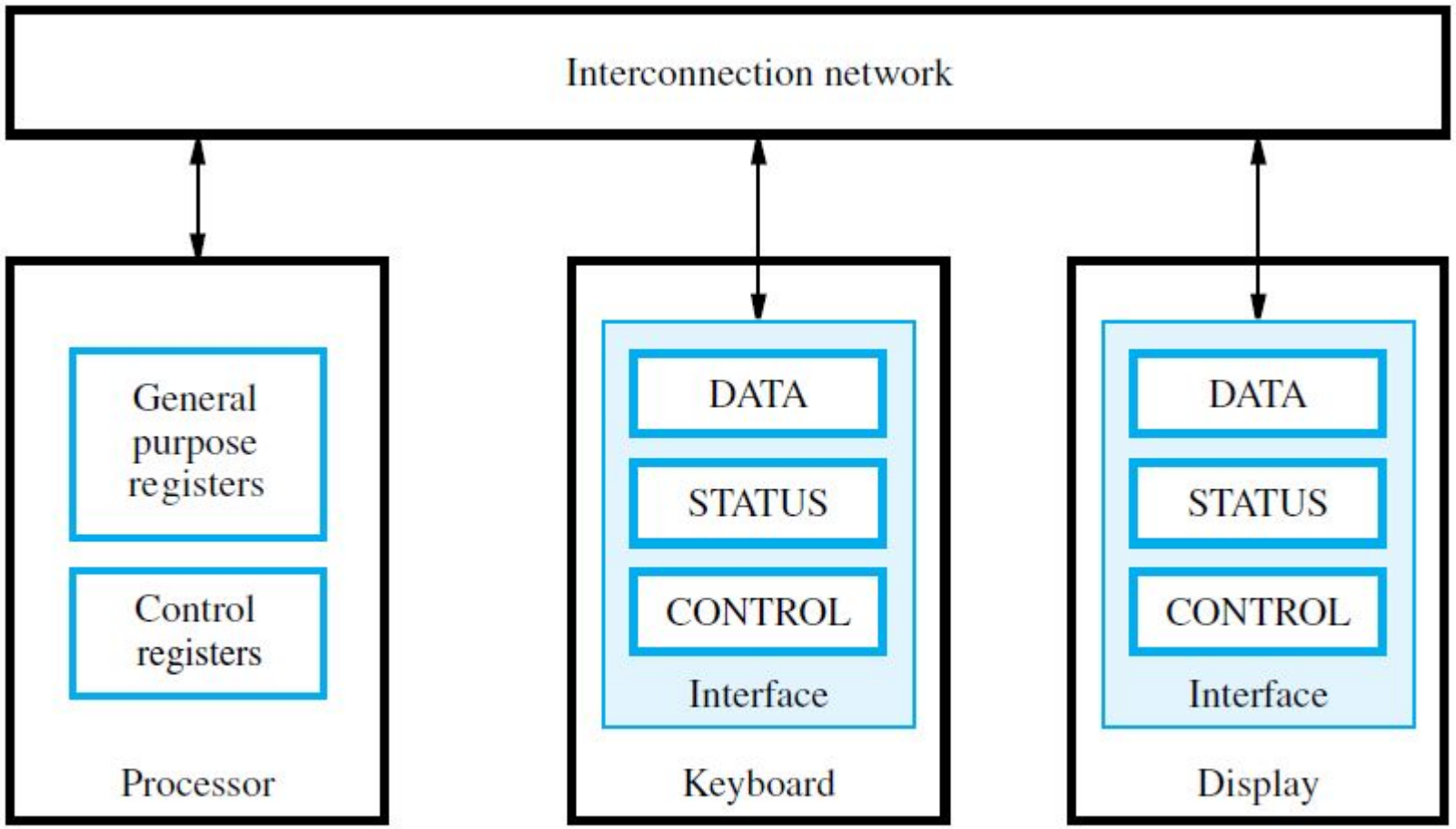


Figure 3.2 The connection for processor, keyboard, and display.

Cont..

- The rate of data transfer from the keyboard to a computer is limited by the typing speed of the user, which is unlikely to exceed a few characters per second.
- The rate of output transfers from the computer to the display is much higher.
- The difference in speed between the processor and I/O devices creates the need for mechanisms to synchronize the transfer of data between them.

Cont..

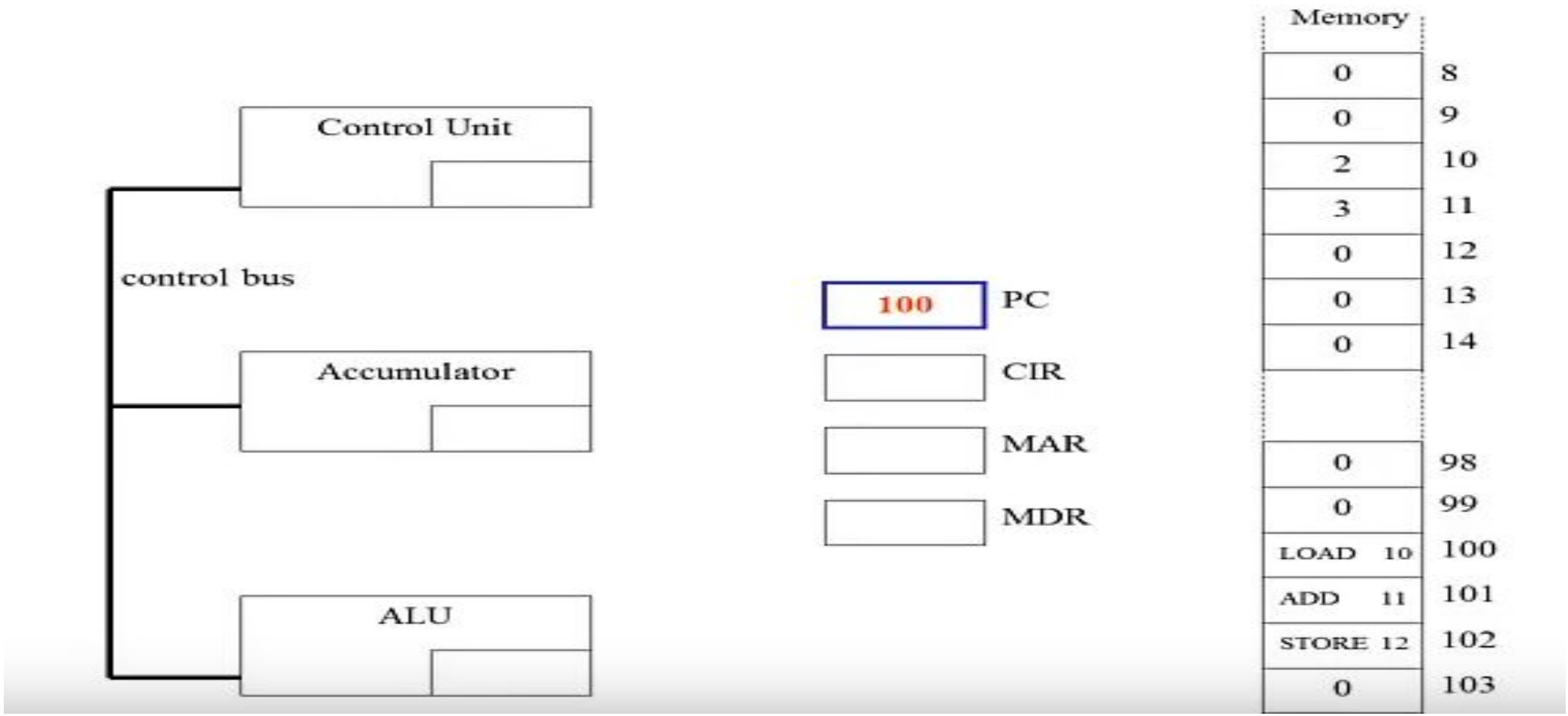
- For example, the contents of the keyboard character buffer KBD_DATA can be transferred to register R5 in the processor by the instruction

  LoadByte R5, KBD_DATA

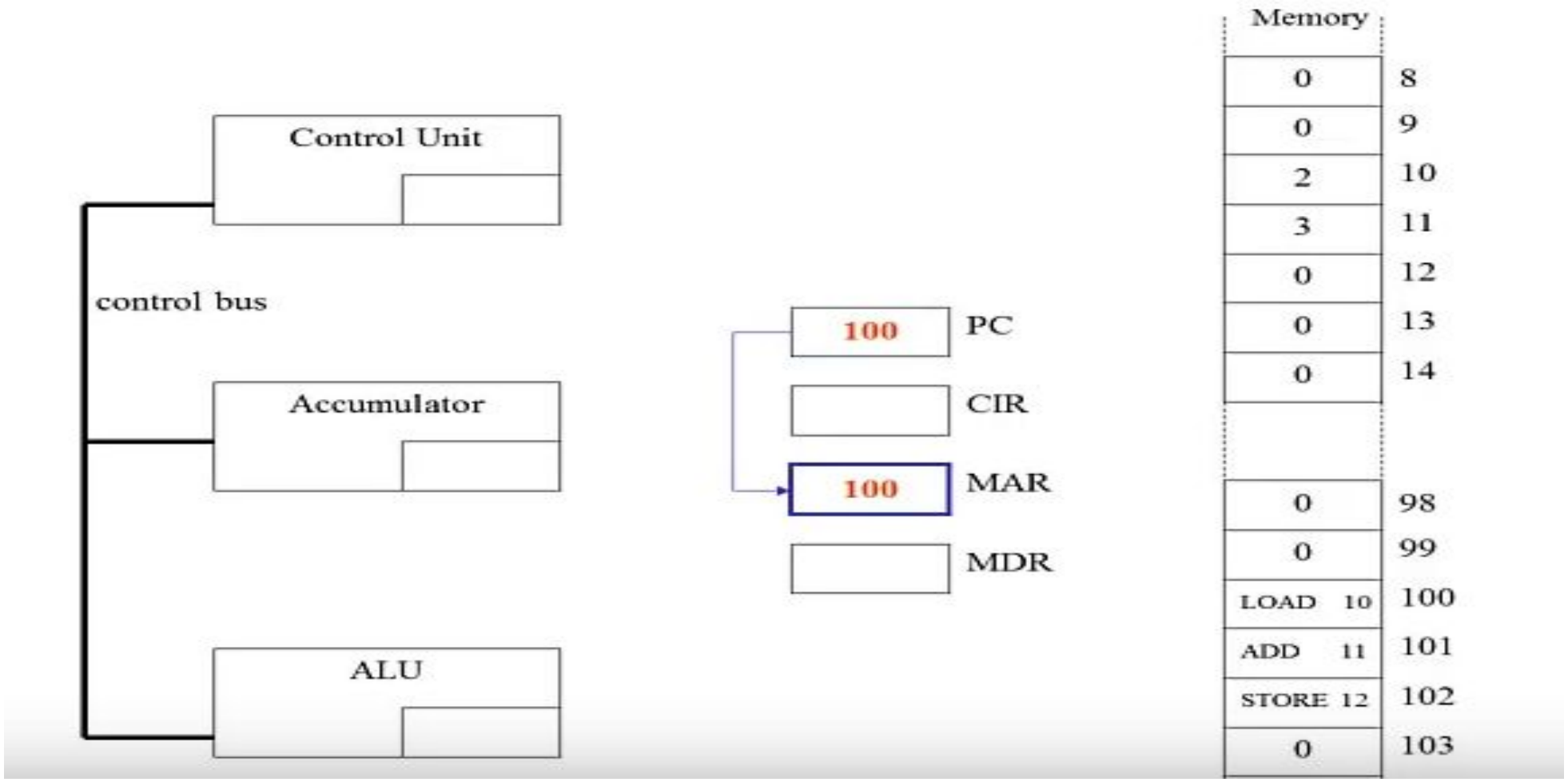
- Similarly, the contents of register R5 can be transferred to DISP_DATA by the instruction

  StoreByte R5, DISP_DATA

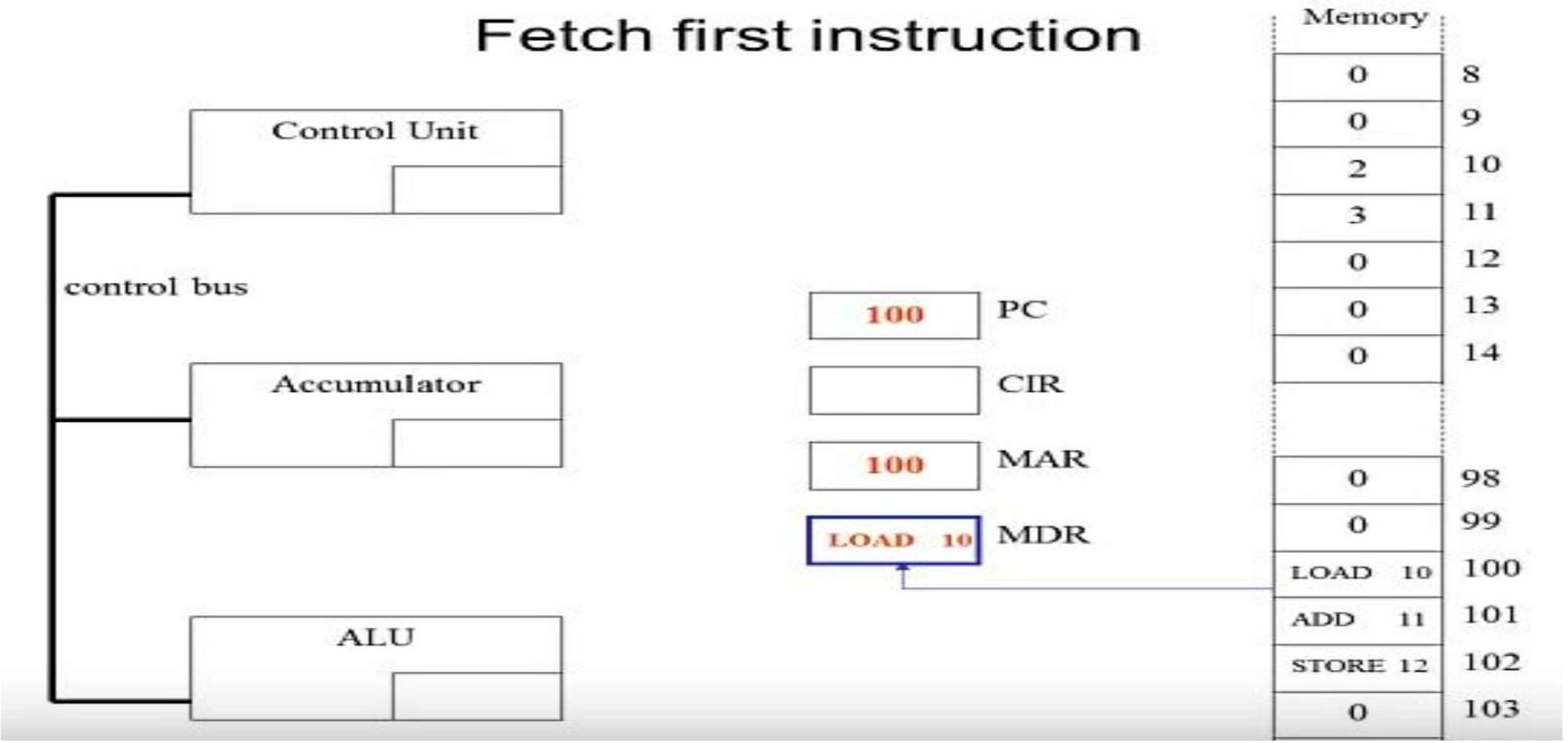
What happen when the program runs



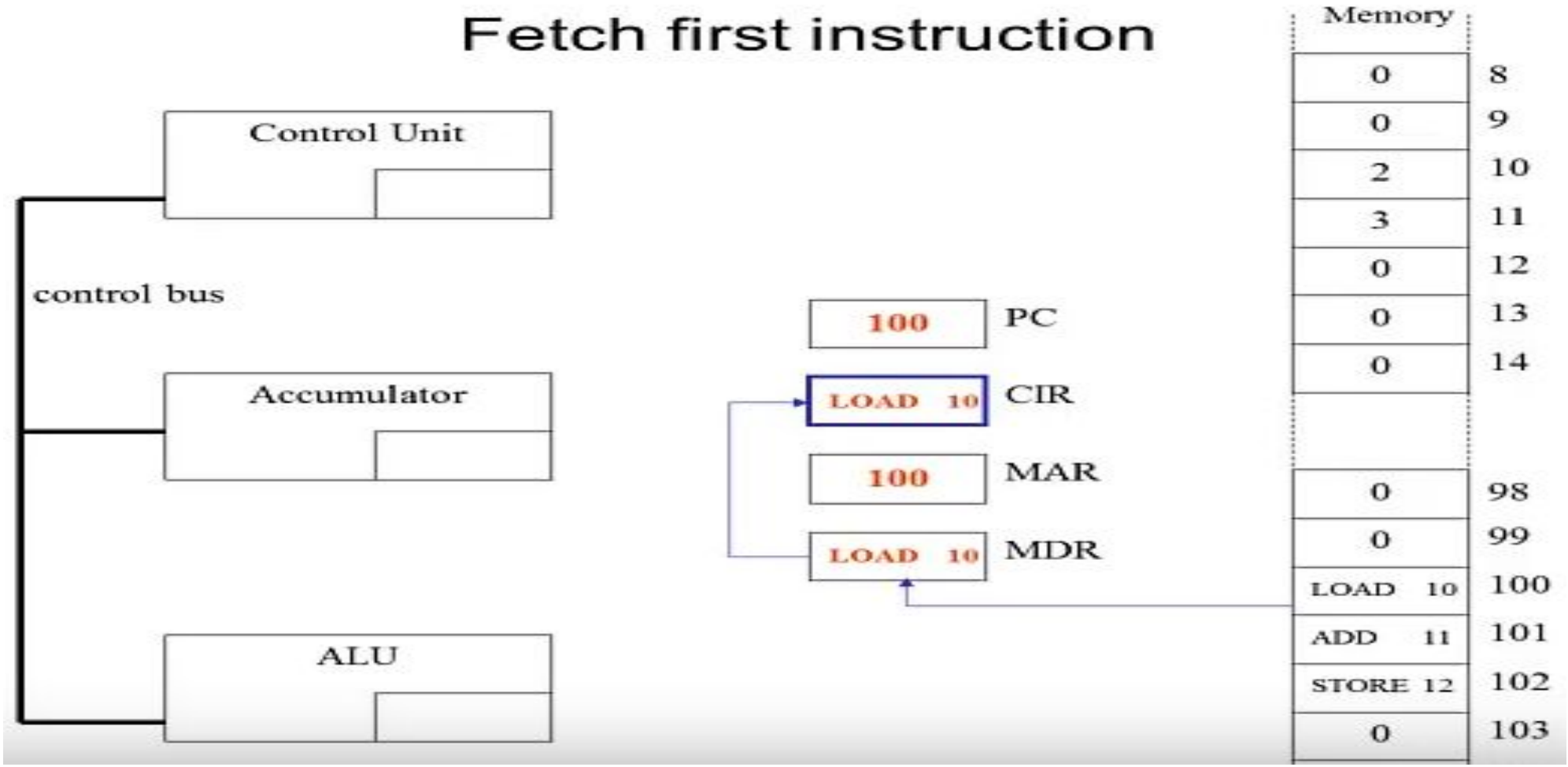
What happen when the program runs



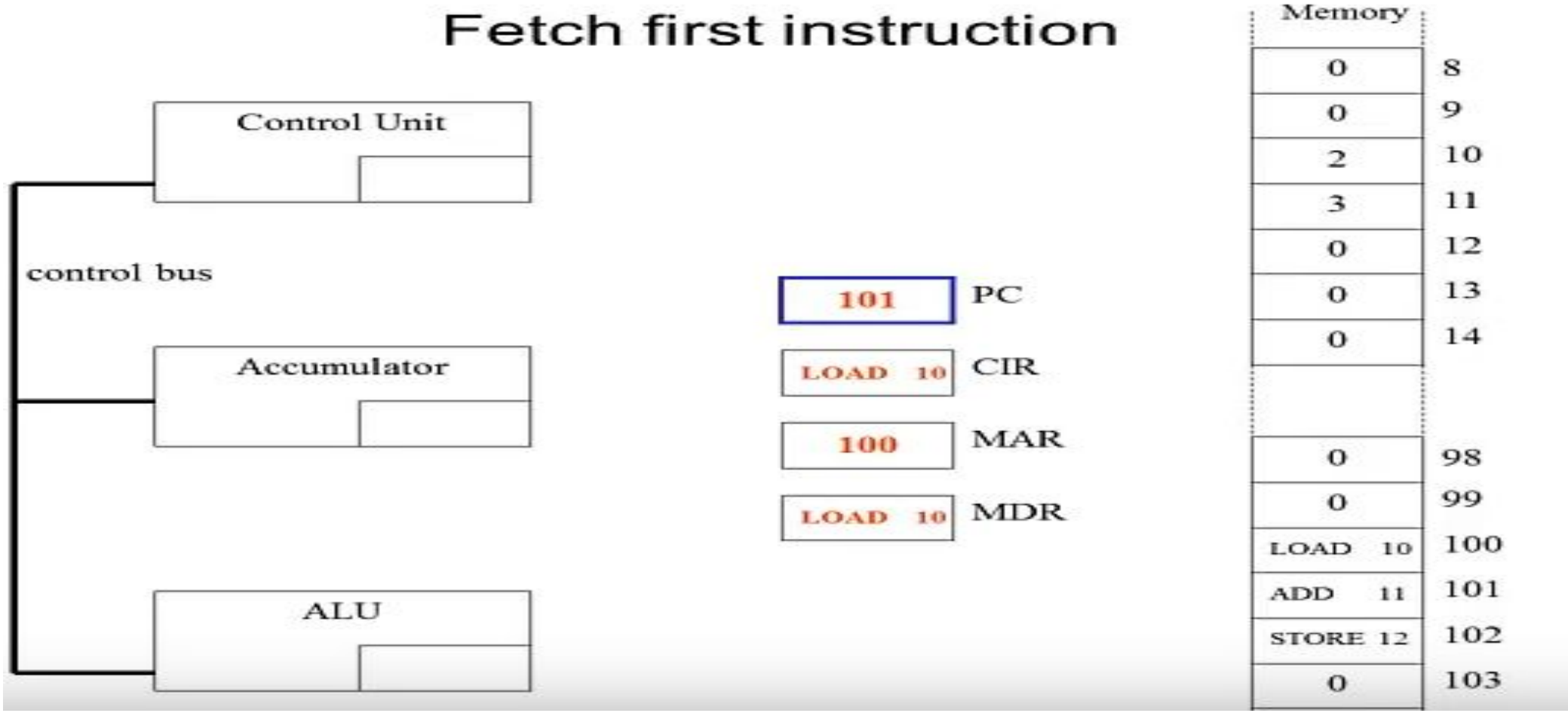
What happen when the program runs



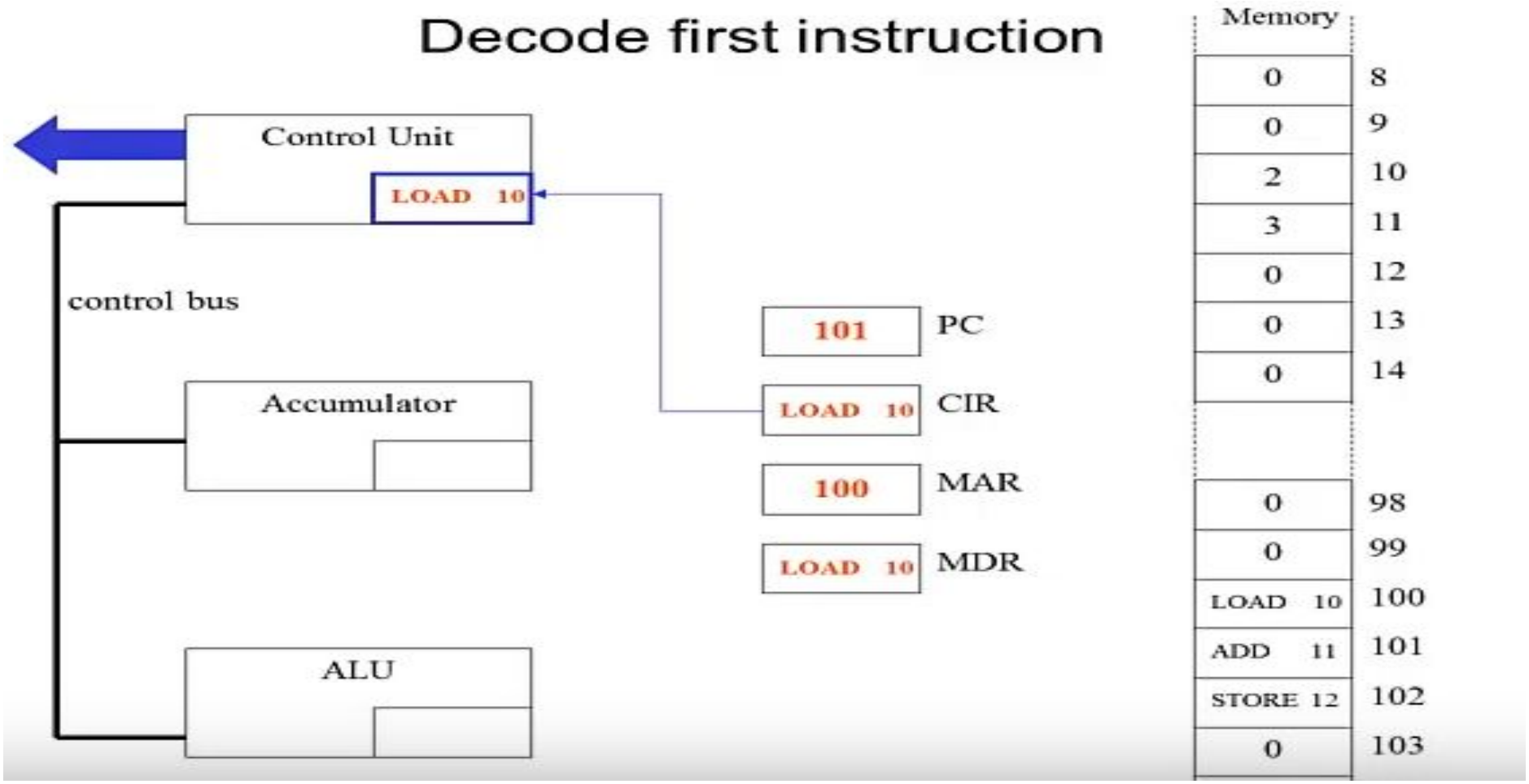
What happen when the program runs



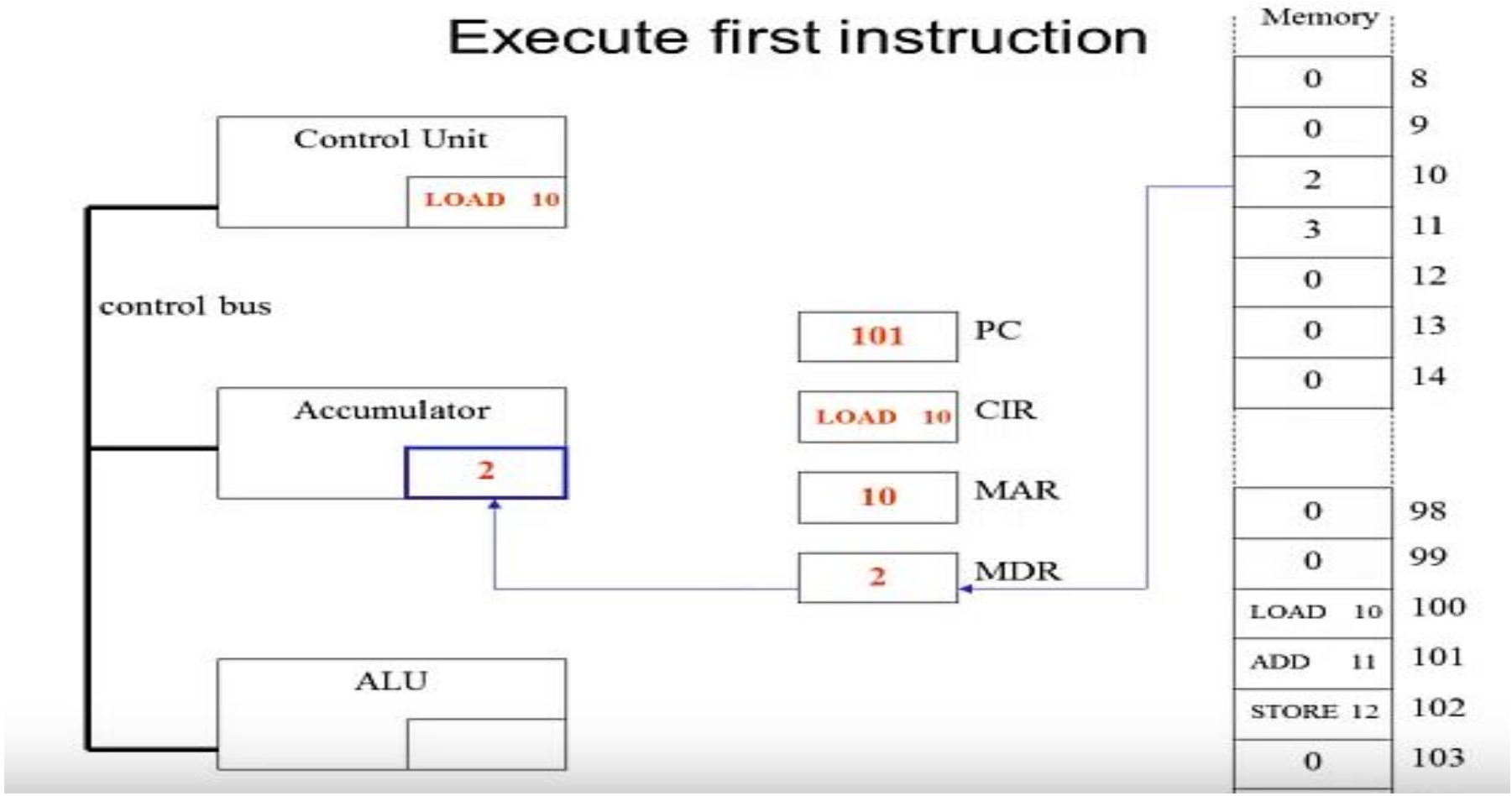
What happen when the program runs



What happen when the program runs



What happen when the program runs

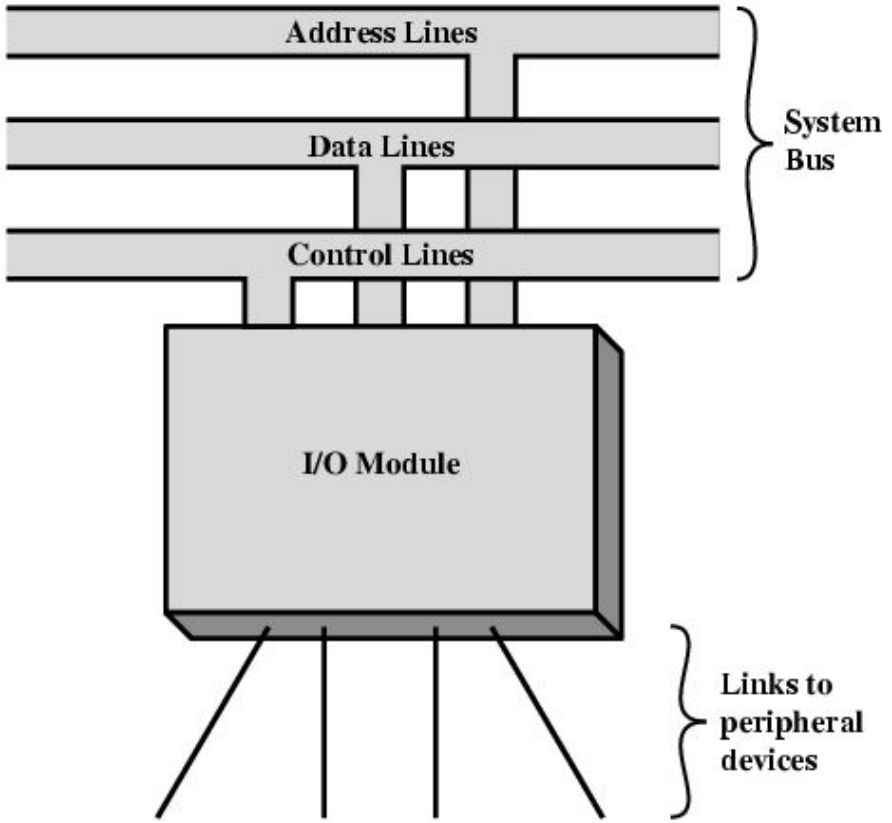


External Devices

- Broadly classify into three categories
  - Human Readable – VDTs and Printers.
  - Machine Readable – magnetic disk and tape system, sensors.
  - Communication – terminals, modem, NIC and other machines.
- Suitable for communicating with computer user – human readable.
- Suitable for communicating with equipment – machine readable.
- Suitable for communicating with remote devices – communication.

External Devices

- Provide a means of exchanging data.
- It connects through a link to an I/O module.
  - Used to exchange control, status, and data between the I/O module and external device.
- External device is referred to us peripheral device or a peripheral.



External Devices

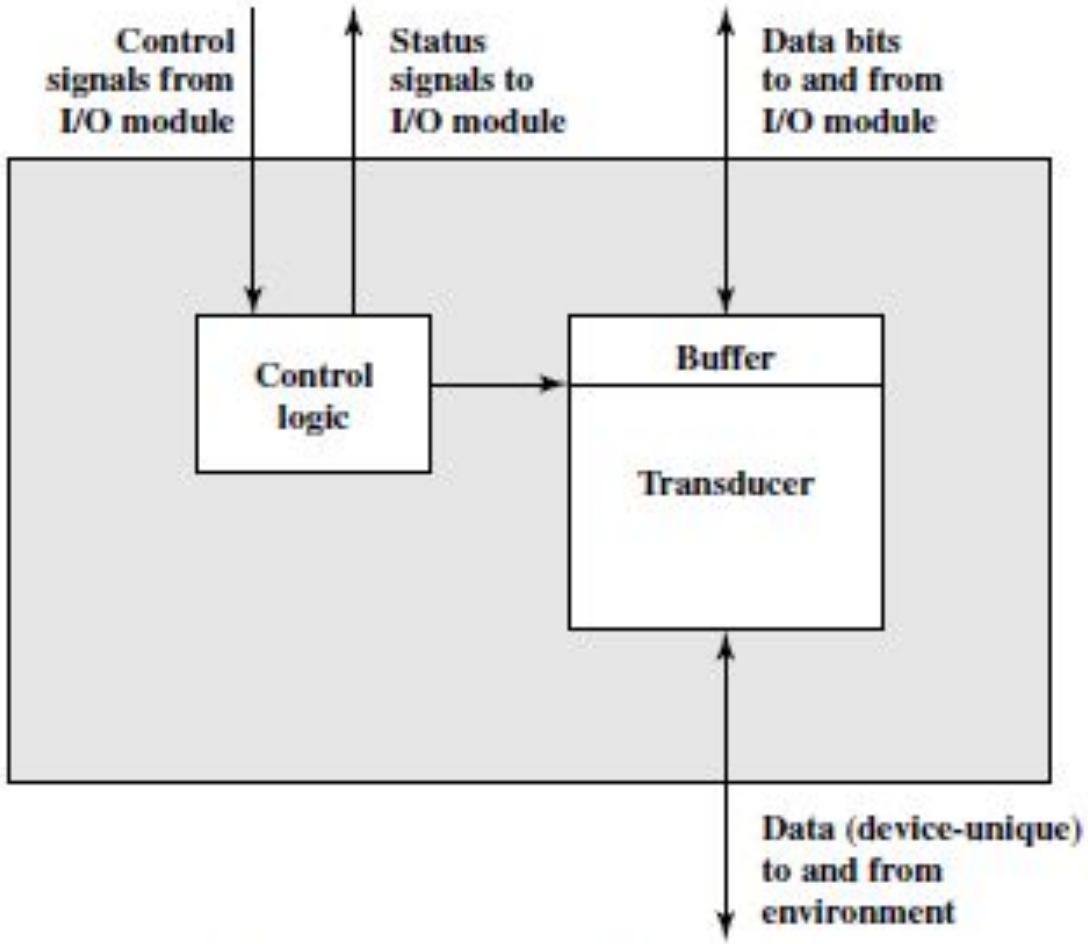


Figure 7.2 Block Diagram of an External Device

External Devices

- Interface to the I/O module is in the form of
  - Control signals, data, and status signals.
- Control signals determine the function that the device will perform.
  - Transfer data to the I/O module, accept data from I/O module, report status, or perform some control function particular to the device.
- Data are in the form of bits to be sent to or received from I/O module.
- Status signals indicate the state of the device.

I/O Modules

Module Functions

- Major functions or requirements for an I/O module
  - Control and Timing
  - Processor communication
  - Device communication
  - Data buffering
  - Error detection

I/O Modules

Control and Timing

- Coordinates the flow of traffic between internal resources and external devices.
- From external device to the processor might involve.
  - Processor questions the I/O module to check the status.
  - The I/O module returns the status.
  - If READY, processor requests the transfer of data, by mean of command to the I/O module.
  - I/O module obtains a unit of data from the external device.
  - The data is transferred from I/O module to the processor.

I/O Modules

Processor Communication

- Command decoding – I/O modules accepts commands from the processor, sent as a signals on the control bus.
- Data – exchanged between the processor and the I/O module over the data bus.
- Status reporting – status of peripheral is reported to processor.
- Address recognition – I/O module must recognize one unique address for each peripheral it controls.

I/O Modules

I/O Module Structure

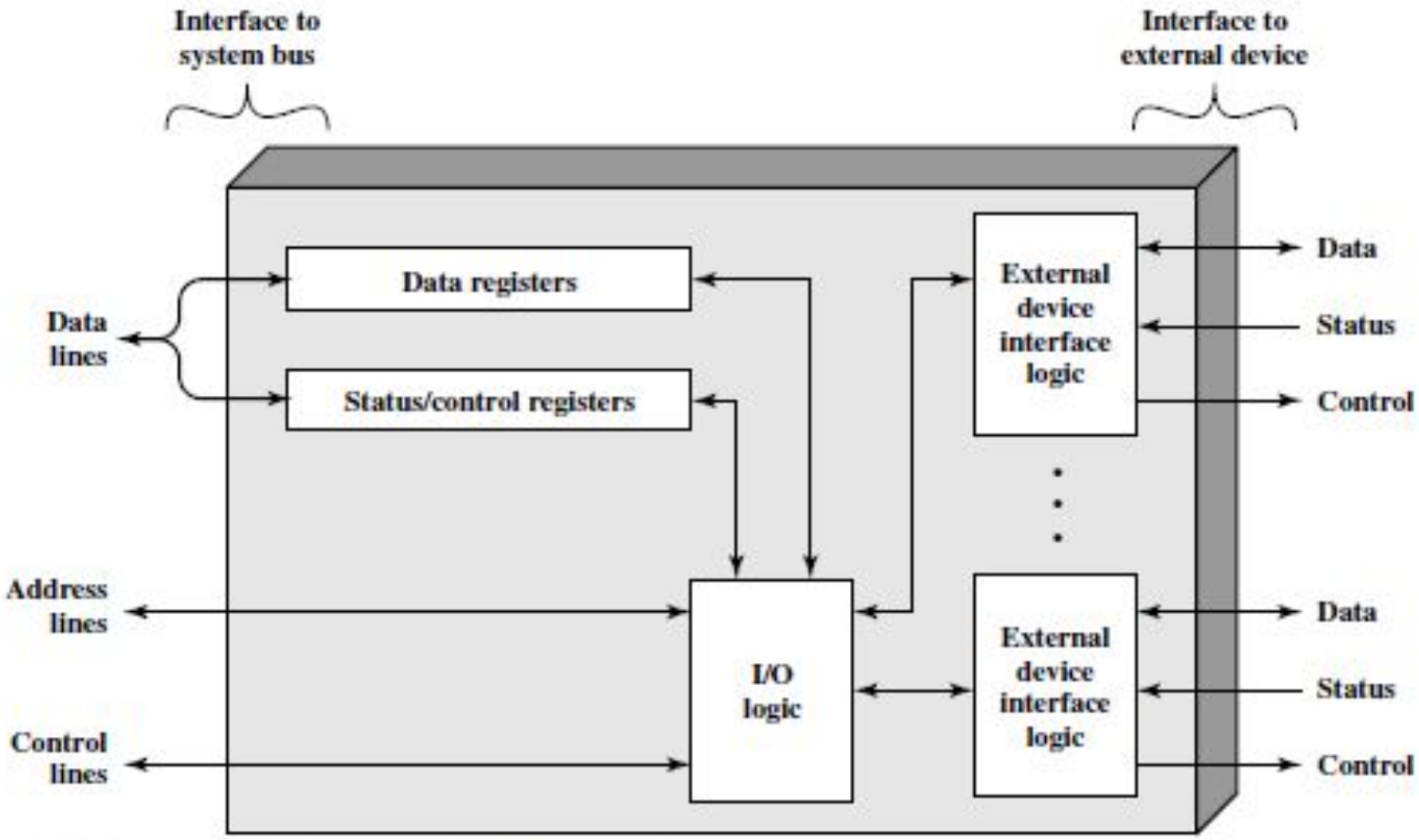


Figure 7.3 Block Diagram of an I/O Module

Programmed I/O

- Three possible techniques for I/O operations.

1. With programmed I/O, data are exchanged between the processor and I/O module.
2. processor executes program that provides direct control of I/O operations.
3. Processor must has to wait for issuing a command until I/O operation is complete.

Programmed I/O

- With interrupt-driven I/O, the processor issues an I/O command
  - Continues its other instructions and is interrupted by the I/O module when completed.
  - In both techniques the processor has to take data in and out from main memory.
- Alternative approach, direct memory access
  - Memory and I/O module directly exchanges the data without the intervention of processor.

Programmed I/O

Table 7.1 I/O Techniques

	No Interrupts	Use of Interrupts
I/O-to-memory transfer through processor	Programmed I/O	Interrupt-driven I/O
Direct I/O-to-memory transfer		Direct memory access (DMA)

Programmed I/O

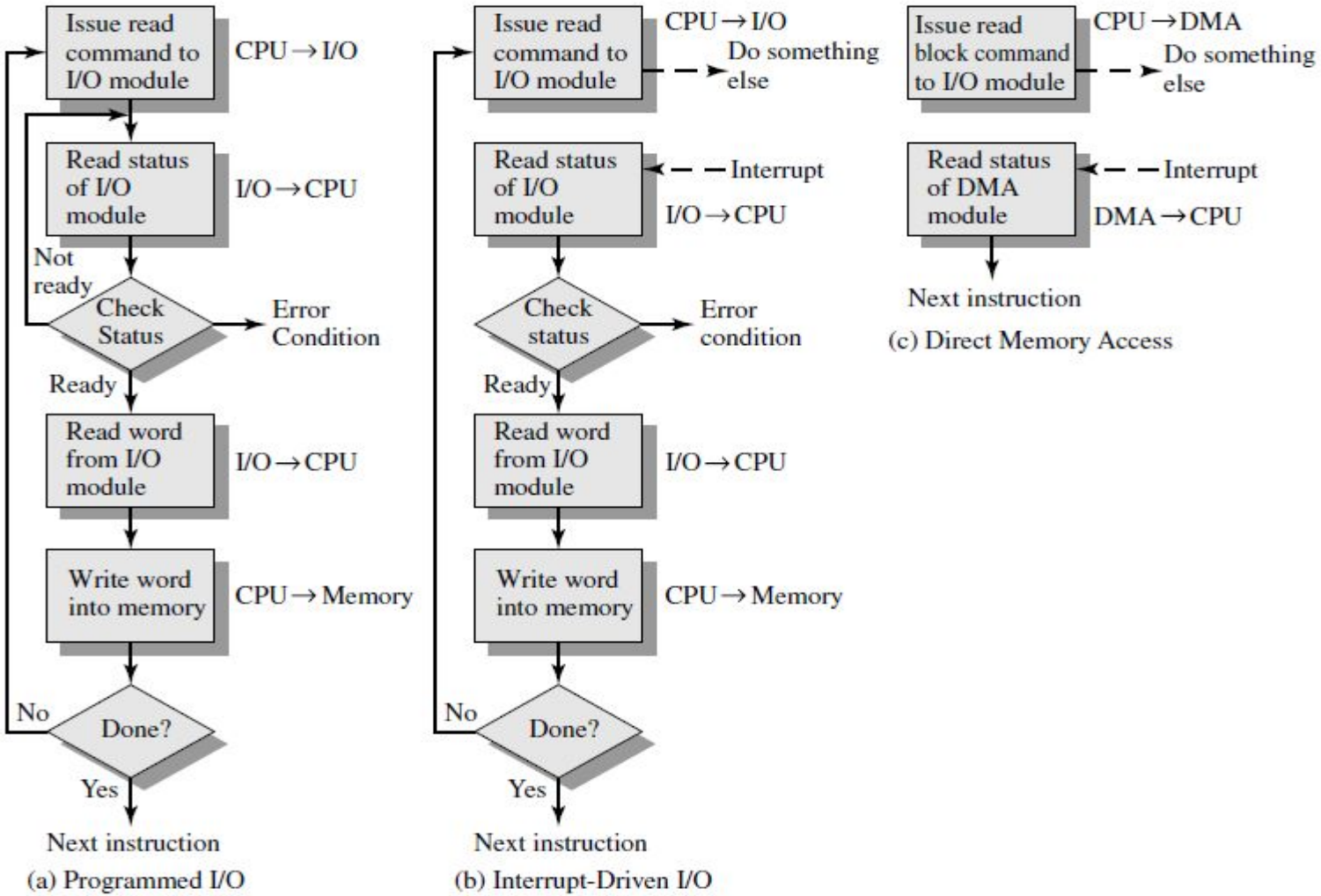


Figure 7.4 Three Techniques for Input of a Block of Data

Programmed I/O

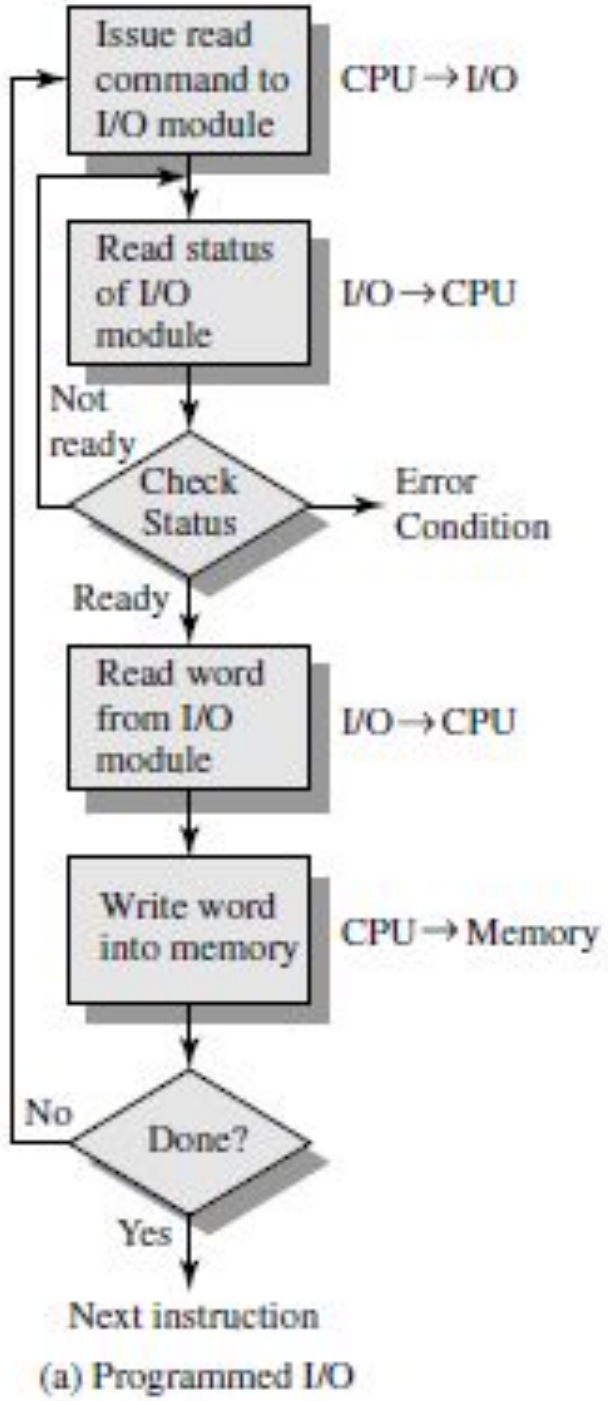
I/O Commands

- To execute I/O related instruction, processor issues
  - Address specifying the I/O module and external device with an I/O command.
- Four types of commands may receive by I/O module
  - Control
  - Test
  - Read
  - Write

Programmed I/O

I/O Commands

- Use of programmed I/O to read in a block of data from a peripheral device into memory.



(a) Programmed I/O

Programmed I/O

I/O Instructions

- Close correspondence between I/O instructions and I/O commands.
  - Instructions are easily mapped into I/O commands.
  - The form of instruction depends on the way in which external devices are addressed.
- Many I/O devices connected though I/O modules to the system.
  - Each device has a unique identifier.
  - Command contains the address of the device.
  - Each module interprets the address lines to determine his command.

Programmed I/O

An Example of a RISC-Style I/O Program

	Move	R2, #LOC	Initialize pointer register R2 to point to the address of the first location in main memory where the characters are to be stored.
	MoveByte	R3, #CR	Load ASCII code for Carriage Return into R3.
READ:	LoadByte	R4, KBD_STATUS	Wait for a character to be entered.
	And	R4, R4, #2	Check the KIN flag.
	Branch_if_[R4]=0	READ	
	LoadByte	R5, KBD_DATA	Read the character from KBD_DATA (this clears KIN to 0).
	StoreByte	R5, (R2)	Write the character into the main memory and
	Add	R2, R2, #1	increment the pointer to main memory.
ECHO:	LoadByte	R4, DISP_STATUS	Wait for the display to become ready.
	And	R4, R4, #4	Check the DOUT flag.
	Branch_if_[R4]=0	ECHO	
	StoreByte	R5, DISP_DATA	Move the character just read to the display buffer register (this clears DOUT to 0).
	Branch_if_[R5]≠[R3]	READ	Check if the character just read is the Carriage Return. If it is not, then branch back and read another character.

Programmed I/O

An Example of a CISC-Style I/O Program

	Move	R2, #LOC	Initialize pointer register R2 to point to the address of the first location in main memory where the characters are to be stored.
READ:	TestBit	KBD_STATUS, #1	Wait for a character to be entered
	Branch=0	READ	in the keyboard buffer KBD_DATA.
	MoveByte	(R2), KBD_DATA	Transfer the character from KBD_DATA into the main memory (this clears KIN to 0).
ECHO:	TestBit	DISP_STATUS, #2	Wait for the display to become ready.
	Branch=0	ECHO	
	MoveByte	DISP_DATA, (R2)	Move the character just read to the display buffer register (this clears DOUT to 0).
	CompareByte	(R2)+, #CR	Check if the character just read is CR (carriage return). If it is not CR, then
	Branch≠0	READ	branch back and read another character. Also, increment the pointer to store the next character.

Summery

- External Devices
- I/O Modules
- Programmed I/O
  - An Example of a RISC-Style I/O Program
  - An Example of a CISC-Style I/O Program

Thank You

For your Patience